

PyPedal — Python Software for Pedigree Analysis

John B. Cole

2025-12-04

1 Preface

1.1 Acknowledgments

PyPedal was initially written to support the author's dissertation research while at [Louisiana State University](#). The initial development was supported in part by a grant from The Seeing Eye, Inc. (Morristown, NJ). The author wishes to thank Paul VanRaden for many helpful suggestions for improving the ability of PyPedal to handle computations in large pedigrees. Additional feedback in the form of bug reports, feature requests, and discussion of computing strategies was provided by Dan Cieslak, Bradley J. Heins (University of Minnesota), Edward H. Hagen (Institute for Theoretical Biology, Humboldt-Universität zu Berlin), Kathy Hanford (University of Nebraska), Thomas Kelly (Department of Animal and Poultry Science, University of Guelph), Thomas von Hassell, Gianluca Saba, and Ken Severin (University of Alaska – Fairbanks). Gregor Gorjanc wrote a blog entry describing [how to install PyPedal on Debian Linux](#). Fernando Perez posted a LaTeX preamble to the NumPy listserver that dramatically improved the PDF version of this manual.

The Newfoundland pedigree was taken from the [Newfoundland Dog database](#) and is used with permission.

The pedigree of European royalty used in the GEDCOM discussion, [ged3.ged](#), was taken from the [Genealogy Forum website](#). It is believed to be in the public domain, and is used with the knowledge of the website administrators.

2 Introduction

i Any sufficiently disguised bug is indistinguishable from a feature. — Rich Kulawiec

2.1 What is PyPedal?

This chapter introduces PyPedal, a Python-language tool for working with pedigrees.

PyPedal (**P**ython **P**edigree **A**nalysis) is a tool for analyzing pedigree files. It calculates several quantitative measures of genetic diversity from pedigrees, including average coefficients of inbreeding and relationship, effective founder numbers, and effective ancestor numbers. Checks are performed catch common mistakes in pedigree files, such as parents with more recent birthdates or smaller ID numbers than their offspring and animals appearing as both sires and dams in the pedigree. Tools for pedigree visualization and report generation are also provided. PyPedal only makes use of information on pedigree structure, not individual genotypes. Genotypes can be assigned to founders for use in gene-dropping simulations to calculate the effective number of founder genomes.

PyPedal is a [Python](#) language module that may be called by programs or used interactively. You must have Python 3.11 (or later) installed in order to use PyPedal. The NumPy module must also be installed in order for you to use PyPedal. In addition, there are several third-party packages used by PyPedal, which are discussed in the chapter on installing PyPedal.

This manual is the official documentation for PyPedal. It includes a tutorial and is the most authoritative source of information about PyPedal with the exception of the source code. The tutorial material will walk you through a series of manipulations of a simple pedigree. All users of PyPedal are encouraged to follow the tutorial with a working installation.

2.2 Features

Key features of PyPedal include:

- Reading pedigree files in user-defined formats;
- Checking pedigree integrity (duplicate IDs, parents younger than offspring, etc.);
- Generating summary information such as frequency of appearance in the pedigree file;

- Reordering and renumbering of pedigree files;
- Computation of the numerator relationship matrix (A) from a pedigree file using the tabular method;
- Inbreeding and relationship calculations for large pedigrees (pedigrees up to 600,000 animals have been processed with PyPedal);
- Computation of coefficients of inbreeding using the Meuwissen and Luo (1992) and modified Meuwissen and Luo (Quaas, 1995, unpublished) algorithms;
- Computation of average total and individual coefficients of inbreeding and relationship;
- Calculation of coefficients of partial inbreeding using an iterative tabular method (Lacy, Alaks, and Walsh 1996; Gulisija et al. 2006);
- Calculation of coefficients of ancestral inbreeding using the methods of Ballou (1997) or Suwanlee et al. (2007);
- Decomposition of A into T and D such that $A = TDT'$;
- Computation of the direct inverse of A (not accounting for inbreeding) using the method of Henderson (1976);
- Computation of the direct inverse of A (accounting for inbreeding) using the method of Quaas (1976);
- Storage of A and its inverse between user sessions as persistent Python objects using the `pickle` module to avoid unnecessary calculations;
- Calculation of theoretical and actual effective population sizes;
- Computation of effective founder number using the exact algorithm of Lacy (1989);
- Computation of effective founder number using the approximate algorithm of Boichard et al. (1997);
- Computation of effective ancestor number using the algorithms of Boichard et al. (1997);
- Selection of subpedigrees containing all ancestors of an animal;
- Identification of the common relatives of two animals;
- Calculation of the inbreeding of offspring from a prospective mating;
- Output to ASCII text files, including matrices, coefficients of inbreeding and relationship, and summary information;
- Importation and exportation of GEDCOM 5.5 files;
- Importation and exportation of GENES 1.20 (dBase III) files;
- Loading pedigrees from, and saving them to, MySQL, Postgres, and SQLite databases;
- Simulation of pedigrees using an algorithm derived from that used in Matvec 1.1a;
- Set operations on pedigrees (unions and intersections);

PyPedal has been used to perform calculations on pedigrees as large as 600,000 animals and has been used in scientific research (Cole, Franke, and Leighton 2004; Cole 2007).

2.3 Where to get information and code

PyPedal and its documentation are available from the [PyPedal 3 Github repository](#).

3 Getting Started

For an example of how to load a pedigree, see Listing 3.1.

Listing 3.1

```
from PyPedal import pyp_newclasses
options = {
    'renumber': 0,
    'pedfile': '../test_files/mrode.ped',
    'pedformat': 'asd',
    'pedname': 'Mrode Pedigree Table 2.1',
    'pedigree_is_renumbered': 1,
    'messages': 'quiet',
}
result = pyp_newclasses.load_pedigree(options)
result.metadata.printme()
```

Metadata for Mrode Pedigree Table 2.1 (../test_files/mrode.ped)

Records:	6
Unique Sires:	3
Unique Dams:	2
Unique Gens:	1
Unique Years:	1
Unique Founders:	2
Unique Herds:	1
Pedigree Code:	asd

4 Methodology

i To iterate is human, to recurse divine. — L. Peter Deutsch

In this chapter, a high-level overview of PyPedal is provided, giving the reader the definitions of the key components of the system. This section defines the concepts used by the remaining sections.

4.1 Reordering and Renumbering

Many computations on pedigrees require that the pedigree be renumbered such that animal IDs are consecutive from 1 to n , where n is the total number of animals in the pedigree. By default, all pedigrees are renumbered at load time. If your pedigree is already renumbered and you don't want it renumbered again you must set the **renumber** option to 0. The renumbering process requires that the pedigree be reordered such that parents always precede their offspring in the list of animal IDs. The actual ID assigned to an animal is of no particular importance, and it is even possible for parents to have larger IDs than their offspring. PyPedal can reorder any pedigree unless there is an error in it that would prevent unambiguously placing parents before offspring. For example, a pedigree containing a keypunch error such that an animal is one of its own grandparents cannot be reordered because there is no way to unambiguously order the animals. The `pyp_utils` module provides two routines for pedigree reordering, `reorder()` and `fast_reorder()`. By default, `reorder()` is used to reorder pedigrees in place. It does this by maintaining a list of animal IDs that have been processed; whenever a parent that isn't in the list of encountered animals is found, the offspring of that parent are moved to the end of the pedigree. This ensures the pedigree is properly sorted such that all parents precede their offspring. Founders are also grouped together at the beginning of the pedigree. This procedure will always correctly reorder a pedigree but it can be quite inefficient as it is similar to an insertion sort, which has a worst-case runtime proportional to n^2 (Cormen et al. 2003).

`fast_reorder()` provides a much faster means of reordering a pedigree, but can incorrectly reorder a pedigree in some cases. When an instance of a `NewAnimal` object is created the `pad_id()` method is called. `pad_id()` uses the animal ID and birth year to form an ID used by `pyp_utils/fast_reorder()` for quick sorting; if your pedigree file is numbered such that offspring always have larger IDs than their parents and your birth years (if provided) are correct (that is, parents always born BEFORE offspring) then

`pyp_utils.fast_reorder()` works as expected. If you don't provide birth years in your pedigree file but your parent IDs are always smaller than your animal IDs, the reordering will be correct. If you do not provide birth years, all animals in the pedigree will be assigned a default value of 1900'. In that case, if parents have IDs larger than that of one or more of their offspring, the pedigree will be incorrectly reordered by `fast_reorder()`. If your pedigree file contains birth years, or you know that parents always have smaller IDs than their offspring, then `fast_reorder()` will correctly reorder your pedigree in linear time. Founders are not guaranteed to be grouped at the beginning of the pedigree when `fast_reorder()` is used; if you're going to calculate coefficients of partial inbreeding (Section `@sec-methodology-partial-inbreeding`) then you should instead use `reorder()` to reorder your pedigree.

The performance difference between the two reordering routines is not very noticeable on pedigrees of a few hundred to a few thousand animals, but is quite dramatic for very large pedigrees. If your pedigree file is already reordered then there's essentially no performance difference between the two. When creating a pedigree file from data stored in a relational database, let the database perform the sort for you by using an `ORDER BY` statement.

4.2 Animal Identification and Cross-References

There are a number of identification attributes associated with animal objects in PyPedal pedigrees. A description of those fields, as well as their default values, is provided in Table `@tbl-objects-animal-objects-new-animal-attributes`. This section describes the data structures provided for mapping between various animal IDs. Table 4.1 lists the four structures provided for ID mapping and lists the default values for pedigrees with integer (`asd` formats) and string (`ASD` formats) IDs. Renumbered PyPedal pedigree objects store animal objects in a list that is typically indexed by renumbered ID. When animal IDs are strings (see Section `@sec-pedigree-format-codes`) they're hashed to an integer and the original ID is stored in a name field. In renumbered pedigrees the original ID is stored and replaced by a renumbered ID.

Once your pedigree is renumbered it's quite easy to see how these maps can be used to convert between various IDs. The maps mean you don't have to worry about renumbered IDs and can continue to think about your animals in terms of their original IDs, whether they be ID numbers or names. Consider the Newfoundland pedigree presented in Figure `@fig-newfoundland-colored-pedigree` – it's much more convenient to think about the dog named Kaptn Kvols von Widdersdorf, rather than the dog whose name was hashed to the ID 5523557808241831142 and renumbered to 48. For example, suppose you wanted to determine his coefficient of inbreeding. It's simple to do using the maps:

`example.namemap['Kaptn Kvols von Widdersdorf']` returns the original ID assigned to the name, while `example.idmap[...]` converts from the original ID to the renumbered ID. This sort

of ID/name mapping is used in a number of places in PyPedal, such as in the three-generation pedigree routine in the `pyp_reports` module.

Note that if an animal has its original ID as its name, which is the default when integral IDs and no animal names are provided, the name is changed to the renumbered ID when the pedigree is renumbered.

Table 4.1: Animal identification and cross-references.

	Key	Value	Key	Value	Key	Value
idmap	animal ID	animal ID	name	name	original ID	renumbered ID
backmap	animal ID	animal ID	name	name	renumbered ID	original ID
namemap	name	animal ID	name	name	name	original ID
name-backmap	animal ID	name	name	name	original ID	name

4.3 Measures of Genetic Variation

Coefficients of inbreeding and relationship (Wright 1922) have been commonly used to describe the genetic diversity in livestock populations (Young and Seykora 1996). Inbreeding coefficients represent an individual's expected genetic homozygosity due to the relatedness of its parents. Coefficients of relationship describe the expected proportion of genes two individuals share due to their relatedness. These are relative measures that depend on such factors as the completeness and depth of pedigrees. Over time, these coefficients change in response to breeding and culling decisions, and they may be used as indicators of the genetic variability of a population. Rapid methods for calculating coefficients of inbreeding and relationship for large populations have been implemented (Wiggans, Van Raden, and Zuurbier 1995).

Populations under study rarely conform to the theory established for the use of coefficients of inbreeding (Wright 1931). Lacy (1989) and Boichard et al. (1997) proposed measures of genetic variation based on ideas from conservation genetics. Lacy (1989) proposed the idea of the number of founder equivalents in assessing populations. A founder is an ancestor whose parents are unknown. If all founders contribute to the population equally, then the founder equivalent is equal to the number of founders. When founders contribute unequally to the population, the number of founder equivalents decreases. Boichard et al. (1997) developed the idea of founder ancestor equivalents, which is the minimum number of ancestors necessary to explain the genetic diversity of the current population. Founder ancestor equivalents account for bottlenecks, unlike founder equivalents, and are more accurate in populations undergoing intense selection. Caballero and Toro (2000) discussed the relationships among these and other measures of diversity in small populations, and demonstrate their use (Toro et al. 2000).

Roughsedge et al. (1999) used average coefficients of inbreeding, average coefficients of relationship, founder equivalent numbers, and founder ancestor numbers to document the decrease in genetic diversity in the British dairy cattle population over the last 25 years. Changes in founder equivalent number and founder ancestor number reflected the use of a small number of influential individuals to improve the genetic merit of that population. Accompanying changes in average inbreeding and relationship did not accurately reflect that loss of diversity. Such results highlight the need for additional tools when assessing complex populations.

4.4 Computational Details

Coefficients of relationship (r_{ij}) and inbreeding (f_i) are calculated using the method of Wiggans et al. (1995). An empty dictionary is created to store animal IDs and coefficients of inbreeding. For each animal in the pedigree, working from youngest to oldest, the dictionary is queried for the animal ID. If the animal does not have an entry in the dictionary, a subpedigree containing only relatives of that animal is extracted and the coefficients of inbreeding are calculated and stored in the dictionary. A second dictionary keeps track of sire-dam combinations seen in the pedigree. If a full-sib of an animal whose pedigree has already been processed is encountered the full-sib receives a COI identical to that of the animal already processed. This approach allows for computation of COI for arbitrarily large populations because it doesn't require allocation of a single NRM of order n^2 , where n is the size of the pedigreed population. In most cases, the NRM for a subpedigree is on the order of 200, although this can vary with species and population data structure.

Average and maximum coefficients of inbreeding are computed for the entire population and for all individuals with non-zero inbreeding. The average relationship among all individuals is also computed. Theoretical and realized effective population sizes, $N_{e(t)}$, and $N_{e(r)}$, were estimated as (Falconer and MacKay 1996):

$$N_{e(t)} = \frac{4N_m N_f}{N_m + N_f}$$

and

$$N_{e(t)} = \frac{1}{2\Delta f}$$

where N_m and N_f are the number of sires and dams in the population, respectively, and Δf is the change in population average inbreeding between generations t and $t+1$. Interpretation of $N_{e(t)}$ can be problematic when Δf is calculated from incomplete or error-prone pedigrees.

4.5 Ancestral Inbreeding

Ballou (1997) described ancestral inbreeding, the probability that an individual inherited an allele that had undergone inbreeding in the past at least once, in a study of purging recessives and inbreeding in conservation genetics. This is a different idea than the usual coefficient of inbreeding as an individual that is not inbred may carry alleles that have been exposed to substantial inbreeding; recall that an individual may have inbred parents, but if the parents are not related to one another then the resulting offspring will not be themselves inbred. It has been proposed that animals deriving from highly inbred lines may be less susceptible to inbreeding depression because deleterious recessive alleles have been purged from the population, but literature reports on extant populations are inconsistent. Ancestral inbreeding is calculated using a recursion equation as:

$$f_a = [f_{a(s)} + (1 - f_{a(s)})f_s + f_{a(d)} + (1 - f_{a(d)})f_d]/2$$

where f_a is the ancestral inbreeding coefficient for an individual, f is the usual coefficient of inbreeding, and subscripts s and d represent sires and dams, respectively. Calculations are from oldest to youngest in the population and require as inputs only coefficients of inbreeding.

Suwanlee et al. (2007) extended the concept by presenting a gene-dropping approach (MacCluer et al. 1986) for calculating ancestral inbreeding, as well as modifying Ballou’s equation to account for non-independence between individual inbreeding coefficients and ancestral inbreeding coefficients. Gene-dropping is also used in PyPedal for calculating founder genome equivalents (see: Section [?@sec-methodology-computation-founder-genome-equivalents](#)), and the code used by the `pyp_metrics.dropped_ancestral_inbreeding()` and `pyp_metrics.effective_founder_genomes()` routines are similar, although the former drops an arbitrary number of unlinked biallelic loci through the pedigree while the latter drops only a single locus.

4.6 Partial Inbreeding

Partial inbreeding coefficients, F_{ij} , measure the probability that the alleles at an arbitrary locus in individual i are identical-by-descent and that the alleles were derived from an allele in founder j [LacyAW1996]; Gulisija et al. (2006) provide an excellent description of the tabular method for calculating F_{ij} . Computational requirements may be high for large pedigrees with a large number of founders because partial kinship matrices are calculated for each founder in the pedigree. The usual coefficients of inbreeding may be obtained by summing the coefficients of partial inbreeding over all founders common to the parents of animal i , that is, $f_i = \sum_j F_{ij}$.

For example, consider the pedigree presented in Figure 2 of Gulisija and Crow (2007). The individual of interest, I , has an inbreeding coefficient of 0.375 and coefficients of partial

inbreeding to founders J, K, and M of 0.21875, 0.09375, and 0.0625, respectively. As asserted, $0.21875 + 0.09375 + 0.0625 = 0.375$.

4.7 Generation Coefficients

{sec-methodology-computation-generation-coefficients}

Generation coefficients are assigned using the method of [Pattie1965]. Founders, defined as individuals with unknown parents, are assigned generation codes of 0. All other animals are assigned generation codes as:

$$GC_o = \frac{(GC_s + GC_d)}{2} + 1$$

where GC_o , GC_s , GC_d represent offspring, sire, and dam codes, respectively.

4.8 Effective Founder Number

{sec-methodology-computation-effective-founder-number}

The effective founder number (f_e) was calculated as:

$$f_e = \frac{1}{\sum p_i^2}$$

where p_i is the proportion of genes contributed by ancestor i to the current population (Lacy 1989). If all founders had contributed equally to the population, then f_e would be the same as the actual number of founders. When founders contribute to the population unequally, f_e is smaller than the actual number of founders. The greater the inequity in founder contributions, the smaller the effective founder number.

A subpedigree approach, similar to that used for calculation of inbreeding (see {sec-methodology-computation-inbreeding} for details), is also used for calculating f_e .

4.9 Founder Genome Equivalents

{sec-methodology-computation-founder-genome-equivalents}

Lacy (1989) also defined the number of founder genome equivalents (f_g) as a measure of genetic diversity. A founder genome equivalent is the number of founders that would produce a population with the same diversity of founder alleles as the pedigree population assuming all

founders contributed equally to each generation of descendants. Founder genome equivalents are calculated as:

$$f_g = \frac{1}{\sum \frac{p_i}{r_i}}$$

where p_i is the proportion of genes contributed by ancestor i to the current population and r_i is the proportion of founder i 's genes that are retained in the current population. Like f_e , f_g accounts for unequal founder contributions. Unlike f_e , f_g also accounts for the fraction of founder genomes lost from the pedigree through drift during bottlenecks. Although f_g is the more accurate description of the amount of founder variation present in a population, it can only be calculated directly for simple pedigrees. For large or complex pedigrees, the number of founder genome equivalents must be approximated based on computer simulation of a large number of segregations through the pedigree. This is done by assigning each founder a unique pair of alleles and randomly transmitting those alleles through the pedigree [ref1719]. The number of founder genome equivalents is similar to the effective founder number, but the former has been devalued based on the proportion of its genome that has probably been lost to drift [ref640].

4.10 Effective Ancestor Number

In populations that have undergone a bottleneck the effective number of founders computed using Lacy's (1989) approach is overestimated. Large contributions made by recent ancestors are more important to the population with respect to the loss of genetic diversity than equal contributions made long ago. Boichard et al. (1997) proposed a second measure of diversity to deal with such situations, the effective number of ancestors (f_a), which considers the genetic contribution of all ancestors in the population, not just founders. The effective number of ancestors treats all ancestors in the population the same way, and is computed as:

$$f_a = \frac{1}{\sum q_i^2}$$

where q_i is the genetic contribution of the i -th ancestor not explained by the previous $i - 1$ ancestors. The ancestors with the greatest contributions are selected iteratively. The number of ancestors with a positive genetic contribution is less than or equal to the actual number of founders.

4.11 Pedigree Completeness

{sec-methodology-pedigree-completeness}

Pedigree completeness [Cassell2003a], the proportion of known pedigree information for an arbitrary number of generations, is computed as:

$$c_p = \frac{a_k}{\sum_{i=1}^g 2^i}$$

where c_p is pedigree completeness and a_k is the number of known ancestors in g generations. The default (which may be changed) is to compute four-generation pedigree completeness. Low c_p indicates that there is little pedigree information available for an individual, which may result in biased estimates of inbreeding and other measures of diversity. Pedigree completeness and ancestor loss coefficients, which are sometimes seen in dog breeding materials, are equivalent measures if the same number of generations were used in the calculations.

Listing 4.1

```
from PyPedal import pyp_newclasses

from PyPedal import pyp_nrm

example = pyp_newclasses.load_pedigree(optionsfile='../test_files/newfoundland.ini')

newf_f = pyp_nrm.inbreeding(example)

print(newf_f['fx'][example.idmap[example.namemap['Kaptn Kvols von Widdersdorf']]])
```

```
[INFO]: Logfile untitled_pedigree.log instantiated.
[INFO]: Preprocessing ../test_files/newfoundland.ped
[INFO]: Opening pedigree file ../test_files/newfoundland.ped
[NOTE]: Added pedigree entry for sire Highland Bear of Pouch Cove
[NOTE]: Added pedigree entry for sire Schooners Yosef of Newton Ark
[NOTE]: Added pedigree entry for sire Amitys Bearfoot of Pouch Cove
[NOTE]: Added pedigree entry for sire Spokinewfs Cariboo Cowboy
[NOTE]: Added pedigree entry for sire Nordbjorns Gringo
[NOTE]: Added pedigree entry for sire Arvals Nordstrands Talisman
[NOTE]: Added pedigree entry for sire Hector van het Heidemeer
[NOTE]: Added pedigree entry for sire Fjordblinks Hakon
[NOTE]: Added pedigree entry for sire Aaron van het Hoogveen
[NOTE]: Added pedigree entry for sire Boedha van het Hoogveen
[NOTE]: Added pedigree entry for sire Enrik von Luxemburg
[NOTE]: Added pedigree entry for sire La Bellas Rigoletto
[NOTE]: Added pedigree entry for sire As-Dur vom Weserwall
[NOTE]: Added pedigree entry for sire Graaf Gabor van Overessinge
[NOTE]: Added pedigree entry for sire Benno von Ammertal
[NOTE]: Added pedigree entry for dam Ad Lib of Pouch Cove
[NOTE]: Added pedigree entry for dam Souvenir of Pouch Cove
[NOTE]: Added pedigree entry for dam Lifebuoys Penny of Bubbeline
[NOTE]: Added pedigree entry for dam Jehaj Vilhelmina
[NOTE]: Added pedigree entry for dam Black Domes Miss Caminickers
[NOTE]: Added pedigree entry for dam Ellis-Bijou of the Thatch Roof
[NOTE]: Added pedigree entry for dam Fenja vom Riesrand
[NOTE]: Added pedigree entry for dam Gracia van de Papenhof
[NOTE]: Added pedigree entry for dam Dukeroys Daphne
[NOTE]: Added pedigree entry for dam Zarah von Luxemburg
[NOTE]: Added pedigree entry for dam Frederikke
[NOTE]: Added pedigree entry for dam Bonny vom Weserwall
[NOTE]: Added pedigree entry for dam Fleur of the Coastguard
[NOTE]: Added pedigree entry for dam Ina vom Forellenhof
[INFO]: Assigning sexes at Fri Dec 5 17:25:09 2025
[NOTE]: pyp_utils/set_sexes() assigning14 a sex to all animals in pedigree Newfoundland Pedigree
[INFO]: Creating pedigree metadata object
[INFO]: Instantiating a new PedigreeMetadata() object...
[INFO]: Naming the Pedigree()...
[INFO]: Assigning a filename...
[INFO]: Attaching a pedigree...
[INFO]: Setting the pedcode...
```

References

- Ballou, J. D. 1997. "Ancestral Inbreeding Only Minimally Affects Inbreeding Depression in Mammalian Populations." *Journal of Heredity* 88: 169–78.
- Boichard, D., L. Maignel, and E. Verrier. 1997. "The Value of Using Probabilities of Gene Origin to Measure Genetic Variability in a Population." *Genetics Selection Evolution* 29: 5–23.
- Caballero, A., and M. A. Toro. 2000. "Interrelations Between Effective Population Size and Other Pedigree Tools for the Management of Conserved Populations." *Genetical Research (Cambridge)* 75: 331–43.
- Cole, J. B. 2007. "PyPedal: A Computer Program for Pedigree Analysis." *Computers and Electronics in Agriculture* 57: 107–13.
- Cole, J. B., D. E. Franke, and E. A. Leighton. 2004. "Population Structure of a Colony of Dog Guides." *Journal of Animal Science* 82: 2906–12.
- Cormen, T. H., C. E. Leiserson, R. L. Rivest, and C. Stein. 2003. *Introduction to Algorithms, Second Edition*. Englewood Cliffs, NJ: Prentice-Hall.
- Falconer, D. S., and F. C. MacKay. 1996. *Introduction to Quantitative Genetics*. 4th ed. Longman.
- Gulisija, D., and J. F. Crow. 2007. "Inferring Purging from Pedigree Data." *Evolution* 61: 1043–51.
- Gulisija, D., D. Gianola, K. A. Weigel, and M. A. Toro. 2006. "Between-Founder Heterogeneity in Inbreeding Depression for Production in Jersey Cows." *Livestock Science* 104: 244–53.
- Henderson, C. R. 1976. "A Simple Method for Computing the Inverse of a Numerator Relationship Matrix Used in Prediction of Breeding Values." *Biometrics* 32: 69–83.
- Lacy, R. C. 1989. "Analysis of Founder Representation in Pedigrees: Founder Equivalents and Founder Genome Equivalents." *Zoo Biology*. 8: 111–23.
- Lacy, R. C., G. Alaks, and A. Walsh. 1996. "Hierarchical Analysis of Inbreeding Depression in *Peromyscus Polionotus*." *Evolution* 50: 2187–2200.
- MacCluer, J. W., J. L. VandeBerg, B. Read, and O. A. Ryder. 1986. "Pedigree Analysis by Computer Simulation." *Zoo Biology*. 5: 147–60.
- Meuwissen, H. E., and Z. Luo. 1992. "Computing Inbreeding Coefficients in Large Populations." *Genetics Selection Evolution* 24: 305–13.
- Quaas, R. L. 1976. "Computing the Diagonal Elements and Inverse of a Large Numerator Relationship Matrix." *Biometrics* 32: 949–53.
- Roughsedge, T., S. Brotherstone, and P. M. Visscher. 1999. "Quantifying Genetic Contributions to a Dairy Cattle Population Using Pedigree Analysis." *Livestock Production Science* 60: 359–69.

- Suwanlee, S., R. Baumung, J. Sölkner, and I. Curik. 2007. "Evaluation of Ancestral Inbreeding Coefficients: Ballou's Formula Versus Gene Dropping." *Conservation Genetics* 8: 489–95.
- Toro, M. A., J. Rodriganez, L. Silio., and C. Rodriguez. 2000. "Genealogical Analysis of a Closed Herd of Black Hairless Iberian Pigs." *Conservation Biology* 14.
- Wiggans, G. R., P. M. Van Raden, and J. Zuurbier. 1995. "Calculation and Use of Inbreeding Coefficients for Genetic Evaluation of United States Dairy Cattle." *Journal of Dairy Science* 78: 1584–90.
- Wright, S. 1922. "Coefficients of Inbreeding and Relationship." *Amer. Nat.* 56: 330–38.
- . 1931. "Evolution in Mendelian Populations." *Genetics* 16: 97–159.
- Young, C. W., and A. J. Seykora. 1996. "Estimates of Inbreeding and Relationship Among Registered Holstein Females in the United States." *Journal of Dairy Science* 79: 502–5.